

Project 6 (Bonus) — LangChainPro-Branching-Parallel-And-Retry-Patterns-For-A-Real-Customer-Support-Pipeline

Title: LangChainPro Branching Parallel And Retry Patterns For A Real Customer Support Pipeline

Type: Single chain pipeline (branching, parallel lookups, retry/fallback — no agent loop, no LangGraph) · **Stack:** Python, LangChain (LCEL), `langchain-openai`, Pydantic, python-dotenv · **Level:** Intermediate **Tagline:** A customer-support ticket pipeline built entirely from real, production-grade LCEL patterns — proving LangChain alone is genuinely enough when a job is multi-step but doesn't need LangGraph's loops, pauses, or long-term memory.

Overview

LangChainPro is a customer-support ticket processing pipeline built entirely with real LCEL patterns — the LangChain-native ways of handling branching, parallelism, retries, and fallbacks, without ever reaching for LangGraph. It classifies an incoming ticket, routes it to the right handler, runs independent account/order lookups concurrently, drafts a reply that survives a flaky model call instead of crashing, and emits both a live-streamed human-readable draft and a structured, machine-readable record from the same underlying answer. Anyone building a fixed-shape, multi-step backend pipeline — support queues, alert triage, form routing — would want something shaped like this instead of defaulting to a heavier agent framework out of habit.

Features

- Routes tickets to billing, technical, or general handlers with `RunnableBranch`, entirely inside one LCEL pipeline
- Runs independent account-info and order-history lookups concurrently with `RunnableParallel`, with measured timing proof it's actually faster than sequential calls
- Hardens the drafting call with `.with_retry()`, scoped to real transient failures only (timeouts, rate limits)
- Falls back to a secondary chain or canned response with `.with_fallbacks()` when retries are exhausted, and flags the result for human review
- Produces a validated, structured `TicketResolution` record (category, summary, confidence, needs-human-review) via `.with_structured_output()`
- Streams the human-readable draft incrementally with `.stream()`, built from the same source as the structured record so the two can never disagree

- Surfaces ambiguous tickets and fallback events as visible, flagged data instead of letting them silently disappear

Tech Stack

- Python
- LangChain (LCEL)
- `langchain-openai`
- Pydantic
- python-dotenv

Architecture

A single straight-line LCEL chain, no agent loop and no LangGraph: a `RunnableBranch` classifier routes each ticket to a billing, technical, or general handler; billing and technical handlers first run a `RunnableParallel` step that fetches account info and order history concurrently; the drafting call in every handler is wrapped in `.with_retry()` and `.with_fallbacks()` for resilience; and a final step feeds the drafted answer into both `.with_structured_output()` (a `TicketResolution` record) and `.stream()` (an incremental human-readable draft), so one shared answer serves both a downstream system and a person reading it live. Every decision is made once, in one direction — nothing loops back or pauses mid-run, which is exactly why LCEL alone is enough here.

Charter (what this project is)

Build **LangChainPro-Branching-Parallel-And-Retry-Patterns-For-A-Real-Customer-Support-Pipeline**: a customer-support ticket processing pipeline built entirely with real LCEL patterns — the LangChain-native ways of handling branching, parallelism, retries, and fallbacks, without ever reaching for LangGraph.

Here's the gap this project fills. No project anywhere in this curriculum actually centers on LangChain itself. Project 1 (Doc04) deliberately uses the raw OpenAI SDK instead — that's Doc04's own teaching point. Project 3 onward is built on LangGraph instead — that's Doc09's territory. Doc05's own Build Task (a small reusable chain module) is the only hands-on LangChain work anywhere in the whole curriculum, and it's just one file. This project is the missing piece: real production LCEL, on its own, proven to be enough for a real job.

That last part matters. This pipeline is genuinely multi-step — classify, look things up, draft a reply, produce a final record — but it never branches back on itself, never loops until some condition is met, and never pauses mid-run for a human to approve something before continuing. The moment any of those three things become real requirements, [Doc09's comparison topic](#) says you'd reach for

LangGraph instead. This project is built to sit right on the LangChain side of that line, on purpose, so you can feel exactly where the line is.

Jump to: [Setup](#) · [Step 1](#) · [Step 2](#) · [Step 3](#) · [Step 4](#)

The Story — what you're actually building

Picture a support ticket landing in a queue: "I was charged twice for my subscription this month." Before any human touches it, a lot of useful work can already happen automatically. First, figure out what kind of ticket this even is — billing, technical, or general — and hand it to the right kind of response logic, the same way a real help desk routes a ticket to the right team. Second, for a billing or technical ticket, pull up the customer's account info and their recent order history at the same time, since neither lookup depends on the other finishing first. Third, draft a reply using all of that — and do it in a way that survives a flaky model call instead of crashing the whole pipeline the moment one API call times out. Fourth, turn that draft into two different things at once: a clean, human-readable message a support agent can read as it's being written, and a structured record — category, summary, confidence, does a human need to check this — that your ticketing system can log and route on.

Every one of those four steps is a real LangChain pattern, not a toy example. `RunnableBranch` is what real routing looks like in LCEL. `RunnableParallel` is what real concurrent lookups look like. `.with_retry()` and `.with_fallbacks()` are what real production resilience looks like, without a single `try/except` scattered through your code. `.with_structured_output()` and `.stream()` are how one chain's output becomes two different useful things for two different audiences (a machine and a person). None of this needs a graph — it's a straight-line pipeline the whole way through, and that's exactly the point.

Before you read further — think about it yourself: Step 1 asks you to build a classifier that routes to exactly 3 handlers. What should happen to a ticket that's genuinely ambiguous — one that reads half like a billing complaint and half like a technical bug report? Should `RunnableBranch`'s default branch silently swallow it into "general," or does something about that ambiguity need to be visible later in the pipeline, not just quietly decided and forgotten? Second: Step 3 asks you to add a fallback for when the primary drafting call fails. A fallback response is, by definition, worse than what you originally wanted. For a billing ticket about a real double-charge, is a cheerful canned "we got your message" reply actually an acceptable thing to send — or is that a case where failing loudly and flagging a human is the more honest choice than silently shipping a lower-quality answer? Sit with both questions before you read the Steps below.

What you're actually building, in one line: a support-ticket pipeline built entirely from LangChain's LCEL pieces, chained together with `|` — no agent loop, no LangGraph.

Why this needs to exist: to prove, with real working code, that a fixed multi-step job (classify, look things up, draft, format) doesn't need a heavier framework just because a heavier one exists — most real backend pipelines are exactly this shape, not a looping agent.

When you'd reach for this at a real job: when a request or ticket has a small, fixed set of categories and paths, and never needs to loop, pause mid-run, or remember state across a restart — support queues, alert triage, form routing.

How it works, mechanically: each step is a `Runnable`, chained with `|`; `RunnableBranch` and `RunnableParallel` handle routing and concurrent lookups, and `.with_retry()` / `.with_fallbacks()` wrap the parts that can fail.

Why not just do it some simpler/different way: you could skip LCEL and write plain Python `if/else` plus `requests.post` calls straight to the model instead. That works, but you lose everything LCEL gives you for free — retries, streaming, and parallel calls all become code you write and debug yourself, and the pipeline stops being one object you can test piece by piece. You could also wrap every model call in a manual `try/except` instead of `.with_retry()` / `.with_fallbacks()` — but then every handler needs its own copy of the same retry logic, and it's easy to end up retrying a bug that will fail the same way every time (exactly the mistake Step 3 warns about), because nothing separates "this failure is temporary" from "this failure is permanent."

Where This Fits

This is a bonus/portfolio project, not part of the main 5-project arc (Projects 1-5). It exists to give plain LangChain — no agent loop, no LangGraph — its own real, hands-on project, since every other project in this curriculum either deliberately skips LangChain (Project 1, per Doc04) or is built on LangGraph instead (Projects 3 onward, per Doc09). This is the LangChain-focused counterpart to those LangGraph-based projects: same idea of "a real multi-step pipeline," built with the tool that fits it, instead of defaulting to the heavier one out of habit.

09_langgraph's "LangChain vs. LangGraph vs. RAG" Core Concepts topic lays out exactly when each tool fits: LangChain (and LCEL specifically) is enough for a straight-line pipeline, even one with a single fixed branching point (`RunnableBranch`) or a single fixed parallel merge (`RunnableParallel`). LangGraph earns its cost once you need a decision that can route back to an earlier step, a loop with a real stopping rule, or a pause-and-resume for a human — none of which this pipeline needs. This project is built specifically to prove that boundary with real, working code, not just to repeat the claim.

Setup (do this once, before Step 1)

```
cd project_13_langchain_patterns
python -m venv .venv
```

```
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install openai langchain langchain-openai langchain-core python-dotenv pydantic
pip freeze > requirements.txt
```

Same `.env` / `.env.example` setup as every other project:

```
OPENAI_API_KEY=sk-...
LOG_LEVEL=INFO
```

This project reuses Doc05's Build Task shape (one small `Runnable` chain module, config-driven model choice) directly — if `prompt | model | parser` doesn't feel comfortable yet, re-read [05_langchain_fundamentals's Core Concepts](#) before Step 1, not during it. You do **not** need `LangGraph`, `langgraph`, or a checkpointer anywhere in this project — if you find yourself reaching for one, stop and re-read **Where This Fits** above.

A Real Example (so this isn't just theory)

Use this scenario, or one close to it, for every step below:

Scenario: SupportCo is a small SaaS company. Its support inbox gets tickets like these: - **Billing:**

"I was charged twice for my Pro subscription this month, order INV-2201. Please refund the duplicate." - **Technical:** "The export button on the dashboard just spins forever and never downloads my file. I'm on Chrome." - **General:** "Do you have a

referral program? I'd like to recommend you to a friend's company." - **Ambiguous**

(for Step 1's edge case): "My invoice looks wrong and the numbers on the dashboard don't match it either — is this a billing mistake or is the dashboard broken?"

Mocked lookups (clearly stand-ins for a real database/API call, never actually implemented for real in this project): `get_account_info(customer_id)` returns a plan name and signup date;

`get_order_history(customer_id)` returns a list of recent invoice IDs. Both are written to pause for about a second, on purpose, to simulate real network latency — that's what makes Step 2's parallel-vs-sequential timing comparison mean something.

Test every step below against the billing, technical, general, and ambiguous tickets above.

Why This Project Is Good for Your Portfolio

- **What problem it solves:** shows you can build a real, multi-step production pipeline in LangChain alone — branching, concurrency, resilience, and dual-format output — without defaulting to a heavier framework just because it's what the rest of your projects use.
- **Why it matters:** most real backend work is exactly this shape — a few fixed decision points, a couple of independent lookups, a call that needs to survive a bad day, and one result that two

different downstream consumers need in two different formats. Knowing this doesn't need LangGraph is as valuable as knowing LangGraph itself.

- **When you'd build something like this at a real job:** any ticket, request, or event pipeline with a small, fixed set of categories and no need to loop, pause, or persist state across restarts — support systems, alert triage, content moderation queues.
- **How it's built:** a `RunnableBranch` classifier and router, a `RunnableParallel` concurrent-lookup step, a drafting chain hardened with `.with_retry()` and `.with_fallbacks()`, and a final step that produces both a `.with_structured_output()` record and a `.stream()`-able human-readable draft from the same underlying answer.

Problems you'll likely run into, and how to fix them:

Problem	Fix
The classifier's default branch quietly swallows every ambiguous ticket into "general," and nobody ever notices	Don't just rely on <code>RunnableBranch</code> 's default — carry the classifier's own confidence (or a simple "did more than one category look plausible" check) through to Step 4's <code>needs_human_review</code> field, so ambiguity becomes visible downstream instead of silently disappearing
<code>RunnableParallel</code> 's two lookups don't actually save any time — the total is still the sum of both	Check the two functions are genuinely independent (neither reads the other's output) and that you're not accidentally calling them sequentially before wrapping them — time both a sequential and a parallel run and compare the numbers directly, don't just assume
<code>.with_retry()</code> is retrying an error that was never going to succeed anyway (like a bad prompt causing a parsing failure)	Scope retries to real transient failures only — a timeout, a rate limit, a connection error — never to a deterministic bug that will fail identically every time; retrying that just burns time and money
The fallback chain fires silently, and a customer gets a degraded, canned response with no record that anything went wrong	Every time <code>.with_fallbacks()</code> triggers, log it and set <code>needs_human_review=True</code> on the final <code>TicketResolution</code> — a fallback should be visible in your data, not just invisible in your users' inboxes
Streaming and the structured record disagree with each other (the streamed draft says one thing, the logged summary says another)	Build both from the exact same drafting-chain output — stream that output to the UI, and separately feed that same text into the structured-extraction step, instead of generating the draft twice with two separate model calls

Built During These Documents

[05_langchain_fundamentals](#) — plus [09_langgraph](#)'s "LangChain vs. LangGraph vs. RAG" comparison topic (see **Where This Fits** above). No LangGraph document is a prerequisite for actually building this project — only for understanding why it's built this way instead of as a graph.

Plan Before You Code

Same process as every project (see [15_five_projects_index](#)): Problem → Requirements → Architecture → Components → Data Flow → Implementation Plan → Coding Tasks. Before you open your editor, write down, in one sentence each, what Step 1 needs to prove, what Step 2 needs to prove, what Step 3 needs to prove, and what Step 4 needs to prove. If you can't state the one thing each step proves, you'll end up building all four patterns at once and never knowing which one is actually doing the work.

How To Build This — Step by Step

Step 1 — A Classification Chain With `RunnableBranch`

Project: LangChainPro-Branching-Parallel-And-Retry-Patterns-For-A-Real-Customer-Support-Pipeline — Step 1 of 4: A Classification Chain With `RunnableBranch`

What this step does: builds a category classifier and uses `RunnableBranch` to route a ticket to one of three draft-response chains — billing, technical, or general — all inside one LCEL pipeline, with no agent loop and no manual `if/elif` routing in your own code. **Why this step matters:**

`RunnableBranch` is LCEL's real answer to "pick one of a few fixed paths based on a decision" — this is the exact capability [Doc09's comparison topic](#) says LangChain handles fine on its own, right up until a decision needs to route back to an earlier step. Feeling where that line sits starts here.

When you'll hit this for real: any pipeline with a small, fixed set of categories and a different next step for each one — support tickets, content moderation queues, form submissions sorted by type.

Read first: [05_langchain_fundamentals Core Concepts](#) — especially "LCEL: chaining pieces together with `|`", [09_langgraph Core Concepts](#) — "LangChain vs. LangGraph vs. RAG" — the "Branching on the model's decision" capability.

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Write `models.py` with a small Pydantic model, `TicketClassification`, holding a `category` field (billing / technical / general) — used with `.with_structured_output()` so the classifier's output is a checked Python object, not a raw string you have to parse yourself. 2. Write `classifier.py`: `build_classification_chain() -> Runnable`, a `ChatPromptTemplate` | `model.with_structured_output(TicketClassification)` chain that reads a ticket's text and returns a `TicketClassification`. 3. Write `handlers.py`: three small draft chains — `billing_chain`, `technical_chain`, `general_chain` — each its own `ChatPromptTemplate` | `model` | `StrOutputParser()`, with a system prompt appropriate to that category (a billing chain that talks about charges and refunds, a technical chain that asks clarifying troubleshooting questions, and so on). 4. Write `router.py`: `build_router_chain() -> Runnable`, combining the above into one `RunnableBranch`. Use `RunnableParallel` to first tag the ticket with its classified category (`RunnableParallel(ticket=RunnablePassthrough(), category=classification_chain)`), then a `RunnableBranch` with one named predicate function

per category (not an inline `lambda` — a plain `def is_billing(inputs): return inputs["category"] == "billing"` is just as correct and much easier for someone else to read later) and `general_chain` as the default (final, un-paired) branch. 5. Test it: run the billing, technical, and general tickets from **A Real Example** and confirm each one reaches the correct handler. Then run the **ambiguous** ticket and look closely at what actually happens — which branch catches it, and whether that's a decision you'd be comfortable defending to a manager, or just wherever `RunnableBranch` 's default happened to send it. You don't have to fix this yet (that's part of what Step 4's `needs_human_review` field is for) — just be honest about what you're seeing.

Your files after Step 1:

```
project_13_langchain_patterns/
├── .env / .env.example
├── config.py                (reused pattern from Doc01: model name, tempera
├── models.py                → TicketClassification (Pydantic): category:
├── classifier.py            → build_classification_chain() -> Runnable
├── handlers.py              → billing_chain, technical_chain, general_c
├── router.py                → build_router_chain() -> Runnable (Runnab
└── main.py                  → runs the 4 example tickets through the
```

Step 2 — Parallel Lookups With `RunnableParallel`

Project: LangChainPro-Branching-Parallel-And-Retry-Patterns-For-A-Real-Customer-Support-Pipeline — Step 2 of 4: Parallel Lookups With `RunnableParallel`

What this step does: adds account-info and order-history lookups for billing and technical tickets, running both at the same time with `RunnableParallel`, merged into one dict the drafting chain can actually use. **Why this step matters:** `RunnableParallel` is LCEL's real, working answer to "run independent things at once and merge the results" — [Doc09's comparison topic](#) calls this out specifically as a case where LangChain doesn't need to hand off to LangGraph at all, because it's one fixed merge point, not a runtime decision about what to run in parallel. **What's new vs. Step 1:** `lookups.py` (two mocked, deliberately slow functions) and `RunnableParallel` wired into the billing and technical branches. **What stays the same:** the `RunnableBranch` router from Step 1 — you're enriching what happens *inside* the billing and technical branches, not changing how a ticket gets routed to them. **When you'll hit this for real:** any time a response depends on two or more independent lookups (a user profile call and a recent-activity call, a pricing call and an inventory call) — running them one after another instead of concurrently is pure wasted latency for no benefit. **Read first:** [09_langgraph Core Concepts — "LangChain vs. LangGraph vs. RAG" — the "Running two things at once and merging results" capability](#), [05_langchain_fundamentals Core Concepts](#) — especially "LCEL: chaining pieces together with `|`".

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Write `lookups.py` : `get_account_info(inputs)` and `get_order_history(inputs)` — plain Python functions, clearly commented as **stand-ins for a real database or API call**, each doing `time.sleep(1)` on purpose to simulate real network latency, then returning a small made-up dict (plan name and signup date for the account; a short list of recent invoice IDs for the order history). 2. Write `research.py` : `build_research_chain()` -> `Runnable`, a `RunnableParallel` wrapping both lookups (using `RunnableLambda`, since they're plain functions, not chains) plus a `RunnablePassthrough()` for the original ticket, so the result dict has `account_info`, `order_history`, and `ticket` all together. 3. Update `handlers.py` 's billing and technical chains so each one is now `research_chain | drafting_prompt | model | StrOutputParser()` — the drafting prompt now reads `{account_info}` and `{order_history}` alongside the ticket text. Leave `general_chain` alone — a general question doesn't need either lookup, and running them anyway would just be wasted work. 4. Time it, honestly: write a small script (or a section of `main.py`) that runs `get_account_info` then `get_order_history` one after another (sequential) and times it, then runs them through `build_research_chain()` (parallel) and times that. Print both numbers. You should see roughly 1 second sequential-doubled vs. roughly 1 second parallel — that's the real, measured proof this step exists to give you, not just a claim in a comment.

Your files after Step 2:

```
project_13_langchain_patterns/
├─ config.py
├─ models.py
├─ classifier.py                (unchanged from Step 1)
├─ lookups.py                  → get_account_info(inputs), get_order_hist
├─ research.py                 → build_research_chain() -> Runnable (Ru
├─ handlers.py                 → billing_chain and technical_chain no
├─ router.py                   (unchanged from Step 1)
└─ main.py                     → runs the 4 example tickets, plus
```

Step 3 — Retry and Fallback for the Drafting Call

Project: LangChainPro-Branching-Parallel-And-Retry-Patterns-For-A-Real-Customer-Support-Pipeline — Step 3 of 4: Retry and Fallback for the Drafting Call

What this step does: hardens the drafting call with `.with_retry()` for transient failures (a timeout, a rate limit) and a genuine `.with_fallbacks()` chain — a cheaper/faster model, or a simpler canned-response template — for when retries run out. **Why this step matters:** a demo pipeline that only ever sees a fast, reliable model call never shows you what a production pipeline actually has to survive — real API calls time out, get rate-limited, and occasionally just fail, and a pipeline that crashes the instant one of those happens isn't production-grade, no matter how good its happy path is. **What's new vs. Step 2:** `drafting.py` (a resilient version of the model call, used inside `handlers.py`) and a fallback chain. **What stays the same:** the router and research chains from Steps 1-2 — this step changes how the model call inside each handler survives failure, not

what gets routed where or what gets looked up. **When you'll hit this for real:** the first time your pipeline is running against real traffic instead of your own test tickets — transient model-call failures are a "when," not an "if," at any real volume. **Read first:** [02_apis_http_json Core Concepts](#) — especially "Retrying with exponential backoff and jitter" (the general idea — LCEL's `.with_retry()` gives you this same behavior built in), [05_langchain_fundamentals Core Concepts](#) — especially "What a library gives you, and what it costs".

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Write `drafting.py`: `build_resilient_model() -> Runnable`, wrapping your `ChatOpenAI` instance with `.with_retry(retry_if_exception_type=..., stop_after_attempt=3, wait_exponential_jitter=True)`, scoped to real transient errors only (a timeout or rate-limit exception type) — never a parsing error or a bad-prompt error, which will fail the exact same way every time no matter how many times you retry it. 2. Write `fallback.py`: a genuine fallback chain, `build_fallback_chain() -> Runnable` — either a `ChatPromptTemplate` | `ChatOpenAI(model=<a cheaper/faster model>)` | `StrOutputParser()` second chain, or (simpler, and worth trying first) a fixed canned-response template filled in with the ticket's category, with no model call at all. 3. Update `handlers.py` so each drafting chain is built as `primary_drafting_chain.with_fallbacks([fallback_chain])`, and make sure whichever path actually ran gets recorded somewhere your code can see afterward (a simple flag or log line saying "fallback used" is enough — you need this for Step 4's `needs_human_review` field). 4. Work through the real design question, and write your answer down as a comment at the top of `fallback.py`: for a billing ticket about an actual double-charge, is your canned fallback response an acceptable thing to actually send, or should a billing-specific failure raise loudly and get flagged for a human instead of silently going out at a lower quality? There's a real, defensible answer either way — the mistake is not deciding on purpose. 5. Test it: force the primary drafting call to fail on purpose (a fake exception raised inside a test double, or a deliberately wrong model name) and confirm the fallback actually produces an answer instead of the whole pipeline crashing. Then confirm a normal, working call never touches the fallback at all — a fallback you can't prove triggers, and can't prove *doesn't* trigger unnecessarily, isn't actually tested.

Your files after Step 3:

```
project_13_langchain_patterns/
├─ config.py
├─ models.py
├─ classifier.py
├─ lookups.py
├─ research.py
├─ drafting.py
├─ fallback.py
├─ handlers.py
├─ router.py
└─ main.py
```

- `build_resilient_model() -> Runnable`, `.with_retry(...)`
- `build_fallback_chain() -> Runnable`, canned
- `billing/technical/general` chains now use (unchanged from Step 1)
- adds a forced-failure test proving th

Step 4 — Structured Output and Streaming the Final Response

Project: LangChainPro-Branching-Parallel-And-Retry-Patterns-For-A-Real-Customer-Support-Pipeline — Step 4 of 4: Structured Output and Streaming the Final Response

What this step does: adds a final step that produces a structured `TicketResolution` (Pydantic) record — category, resolution summary, confidence, whether it needs human review — and, separately, supports streaming the human-readable draft to a UI in real time, both built from the same underlying drafted answer. **Why this step matters:** "one answer" from an LLM is often genuinely needed in two different shapes at once — a machine-readable record for logging, routing, and metrics, and a human-readable stream for a person watching it arrive. Building both from one shared source, instead of two separate model calls, is the difference between a coherent pipeline and two answers that can quietly disagree with each other. **What's new vs. Step 3:**

`resolution.py` (the structured-output step) and a `stream_draft()` function. **What stays the same:** everything from Steps 1-3 — the drafted text this step works from is exactly what the (now retry-and-fallback-hardened) drafting chain already produces. **When you'll hit this for real:** any user-facing feature where a person needs to see a response appear live, while some other part of your system also needs a clean, structured record of what happened — support tools, chat UIs backed by a logging/analytics pipeline, anything with both a person and a downstream system consuming the same answer. **Read first:** [05_langchain_fundamentals Core Concepts](#) — especially "Output parsers: turning raw text into a real Python object" — and its "Intermediate — swap in a structured parser" exercise (`.with_structured_output()`) at [#ex-structured_parser_swap](#) , [04_openai_api's "Real-world — make it feel alive with streaming" exercise](#).

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Write `models.py`'s second Pydantic model, `TicketResolution: category: str, resolution_summary: str, confidence: float, needs_human_review: bool`. 2. Write `resolution.py: build_resolution_chain() -> Runnable`, a `ChatPromptTemplate | model.with_structured_output(TicketResolution)` chain that reads the ticket text plus the drafted answer from Step 3, and produces a `TicketResolution`. Set `needs_human_review=True` whenever the classifier's confidence was low, the ticket was the ambiguous ("could be billing or technical") case from Step 1, or Step 3's fallback chain fired — this is where all three of those earlier "should a human see this" moments finally become one real, visible field, instead of getting silently lost along the way. 3. Write `stream_draft(ticket_input)` in `resolution.py` (or its own `streaming.py`): a small function that calls `.stream()` on the (Step 3, resilient) drafting chain and yields each piece of text as it arrives, so a caller can print or forward it live instead of waiting for the whole answer. 4. Update `main.py` to run one ticket end to end: stream the human-readable draft to the terminal piece by piece (proving `.stream()` really is incremental, not the whole string dumped at once with a fake typing effect), and separately print the structured `TicketResolution` for that same ticket. 5. Test it: confirm the streamed draft's actual content and the structured record's `resolution_summary` genuinely describe the same underlying answer — not two different model calls that happened to both run and could disagree. Also confirm the ambiguous ticket from **A Real Example** ends up with `needs_human_review=True`.

Your files after Step 4 (final):

```

project_13_langchain_patterns/
├─ config.py
├─ models.py                → TicketClassification, TicketResolution
├─ classifier.py
├─ lookups.py
├─ research.py
├─ drafting.py
├─ fallback.py
├─ handlers.py
├─ router.py
├─ resolution.py           → build_resolution_chain() -> Runnable, stream
├─ main.py                 → full pipeline: classify -> route -> resolve
└─ test_pipeline.py        → routing (incl. ambiguous case), parallel

```

Final Deliverable: LangChainPro-Branching-Parallel-And-Retry-Patterns-For-A-Real-Customer-Support-Pipeline — a real customer-support ticket pipeline built entirely from LCEL:

`RunnableBranch` routing, `RunnableParallel` concurrent lookups, `.with_retry()` plus `.with_fallbacks()` resilience, and a final step that produces both a structured `TicketResolution` and a streamed human-readable draft from one shared answer.

Why this genuinely doesn't need LangGraph (said plainly): every decision in this pipeline is made once, in one direction — classify, then route, then (maybe) research, then draft, then finish. Nothing here ever needs to go back to an earlier step, run an unknown number of times until some condition is met, or freeze mid-run waiting on a person. The moment a real requirement forced any of those three things — "let the customer re-answer if the bot misunderstood," "keep refining the draft until a reviewer approves it," "pause here until a human clicks approve" — that would be the honest signal to rebuild this as a LangGraph graph instead, exactly as [Doc09's comparison topic](#) describes. This project's whole point is proving that until that happens, LangChain alone is not a compromise — it's the right-sized tool.

Checklist Before You Call This Done

- [] `RunnableBranch` correctly routes the billing, technical, and general example tickets to the right handler
- [] You can explain exactly what happens to the ambiguous ticket, and why that's an acceptable (or flagged) outcome, not just "whatever the default branch did"
- [] `RunnableParallel`'s two lookups are proven, with real timing numbers, to run concurrently — not just assumed to
- [] `.with_retry()` is scoped to real transient failures only, never to a deterministic bug that would fail the same way every retry

- [] `.with_fallbacks()` is proven to actually fire on a forced failure, and proven to *not* fire on a normal, working call
- [] You wrote down, on purpose, whether your fallback response is an acceptable degraded answer or something that should fail loudly instead — for at least the billing case
- [] The final `TicketResolution` is a real, validated Pydantic object, and `needs_human_review` is actually `True` for the ambiguous ticket and for any run where the fallback fired
- [] `.stream()` prints the draft incrementally (visibly piece by piece), not the full string printed all at once
- [] No `LangGraph` import anywhere in this project — if you found yourself wanting one, that's Doc09's comparison topic proving its point, not a bug to fix here

Status

Not started. Track your progress in [../PROGRESS.md](#).

Stuck? Ask for **Hint 1** or **Hint 2** about the exact part you're stuck on. Only ask for the full code if you say "**Show me the solution.**"

Part of a 13-project multi-agent AI engineering curriculum. See the [full curriculum](#) for the complete learning path and all other projects.